

Article

Efficient Low-Precision GEMM on Ascend NPU: HGEMM's Synergy of Pipeline Scheduling, Tiling, and Memory Optimization

Erkun Zhang ^{1,2,*} , Pengxiang Xu ²  and Lu Lu ^{2,3,*} ¹ School of Future Technology, South China University of Technology, Guangzhou 510641, China² Pengcheng Laboratory, Shenzhen 518071, China³ School of Computer Science & Engineering, South China University of Technology, Guangzhou 510006, China

* Correspondence: ftcallmez@mail.scut.edu.cn (E.Z.); lul@scut.edu.cn (L.L.)

Abstract

As one of the most widely used high-performance kernels, General Matrix Multiplication, or GEMM, plays a pivotal role in diverse application fields. With the growing prevalence of training for Convolutional Neural Networks (CNNs) and Large Language Models (LLMs), the design and implementation of high-efficiency, low-precision GEMM on modern Neural Processing Unit (NPU) platforms are of great significance. In this work, HGEMM for Ascend NPU is presented, which enables collaborative processing of different computation types by Cube units and Vector units. The major contributions of this work are the following: (i) dual-stream pipeline scheduling is implemented, which synchronizes padding operations, matrix–matrix multiplications, and element-wise instructions across hierarchical buffers and compute units; (ii) a suite of tiling strategies and a corresponding strategy selection mechanism are developed, comprehensively accounting for the impacts from M , N , and K directions; and (iii) *SplitK* as well as *ShuffleK* methods are raised to address the challenges of memory access efficiency and AI Core utilization. Extensive evaluations demonstrate that our proposed HGEMM achieves an average $3.56\times$ speedup over the CATLASS template-based implementation under identical Ascend NPU configurations, and an average $2.10\times$ speedup relative to the cuBLAS implementation on Nvidia A800 GPUs under general random workloads. It also achieves a maximum computational utilization exceeding 90% under benchmark workloads. Moreover, the proposed HGEMM not only significantly outperforms the CATLASS template-based implementation but also delivers efficiency comparable to the cuBLAS implementation in OPT-based bandwidth-limited LLM inference workloads.

Keywords: GEMM; ascend NPU; LLM inference; pipeline scheduling

Academic Editor: Paolo Bellavista

Received: 22 November 2025

Revised: 31 December 2025

Accepted: 7 January 2026

Published: 8 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the [Creative Commons](https://creativecommons.org/licenses/by/4.0/)[Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

General Matrix Multiplication, or GEMM, is one of the most widely used high-performance kernels and applied in a variety of engineering fields [1–3]. Recent years have observed that AI is drawing more and more popularity, and GEMM also acts a keystone in many AI applications. Some typical applications are Convolution Neural Networks (CNNs) and Large-scale Language Models (LLMs) in Natural Language Processing (NLP) tasks.

With increasing popularity of research and application on a variety of AI tasks, a series of Basic Linear Algebra Subprograms (BLAS) libraries (e.g. cuBLAS [4], rocBLAS [5], BLIS [6], and MAGMA [7]) and corresponding kernel-level templates (e.g., CUTLASS [8,9])

and CATLASS [10,11]) have been developed that ease the use of the hardware design and its ability in computations. There has already been a variety of studies focusing on the utilization of half-precision GEMM (HGEMM) for CNNs and LLMs [12,13], where matrices are relatively small and difficult to utilize hardware ability without dedicated optimization strategies. An example is the Llama-2-7B model [14], where the matrix size is 2048, 128, and 2048 along M , N , and K directions, respectively, for the matrix multiplication of Query (Q) and Key (K) in self-attention calculation through the implementation of GEMM. In addition, to further accelerate trainings on LLMs as well as enhance memory savings, there are efforts of applying lower-precision GEMMs including INT8 [15], FP6, [16] and INT4 precisions [17,18]. A prevalent strategy involves coalescing multiple lightweight operations into a single, computationally intensive kernel—a design choice that has driven the widespread adoption of batched GEMM. Indeed, extensive empirical evidence has validated that batched GEMM yields substantial performance gains for workloads dominated by small-matrix multiplications [19–21]. Such operations are ubiquitous across domains including tensor computation, deep learning, and finite element analysis [22,23].

The architecture of Graphics Processing Unit (GPU) determines that it is skilled for parallel processing and graphic rendering, and is nowadays commonly used to accelerate AI tasks where high-density computational loads are involved [19,24,25]. On the other hand, Neural Processing Unit (NPU) can be considered to be a simplified GPU, which preserves the Stream Multiprocessor (SM) while it removes rendering unit. Although NPU is designed to accelerate deep learning algorithms initially, it still delivers significantly higher performance for a wide variety of AI tasks compared to CPU and even GPU in certain scenarios today, and greatly improves cost–energy performance [26–28]. With decreased power consumption and simplified structure leading to a shrink in dimension, NPU has already been applied to portable devices such as smart phones to cope with the increasing demand on mobile Deep Neural Network (DNN) applications [28,29].

It is worth pointing out that when it comes to Ascend NPU, HGEMM is the base of quantized lower-precision GEMMs since FP16 precision is the lowest precision supported by the Cube unit embedded in hardware architecture. Moreover, the separated layout of Cube and Vector units brings excessive memory movements across the NPU’s hierarchical memory and disrupts computation pipelines unless there are well-orchestrated overlaps, due to the fact that element-wise operations (e.g., quantization and dequantization) can only be conducted on a Vector unit, while a Cube unit provides high computational ability that handles high-efficiency matrix–matrix multiplications.

To address these challenges, we design and optimize HGEMM on Ascend NPU, which makes use of both Cube and Vector units to handle matrix–matrix multiplication and element-wise accumulation, respectively. By leveraging the Single-Program-Multiple-Data (SPMD) structure and dual-stream AIC/AIV pipelines (Ascend Instruction Cube/Vector streams, for parallel Cube and Vector operations) of Ascend NPU, our proposed HGEMM strikes high computational efficiency and minimizes excessive latency from memory access. To sum up, the main contributions of this study are as follows:

- **AIC/AIV Dual-Stream Pipeline Scheduling.** Both AIC and AIV streams are deployed, with elaborately orchestrated pipelining adopted to achieve high-efficiency HGEMM, which synchronizes padding operations, matrix–matrix multiplications, and element-wise instructions across hierarchical buffers and compute units.
- **Self-Adaptive Tiling Design.** A suite of tiling strategies along with a corresponding strategy selection mechanism is proposed to determine the optimal tiling strategy for each specific matrix size. This mechanism balances the reduction in accumulation loops in the K direction and the maximization of hardware utilization in the M and

N directions. Such an approach is also applicable to other SPMD-based GPU, NPU, or TPU architectures aiming to exploit hardware capabilities.

- **Memory Access and Hardware Occupancy Utilization.** The *ShuffleK* and *SplitK* methods are proposed to address the challenges of memory access efficiency and AI Core occupancy, respectively. The *ShuffleK* method alleviates the occurrence of serial access to shared and global memory, while the *SplitK* method resolves insufficient workload in small matrix scenarios, thereby improving HGEMM efficiency across a broad range of workloads.

The rest of the paper is organized as follows. Section 2 demonstrates the motivation of this research and related works. Section 3 introduces a GEMM routine as well as Ascend NPU architecture. Section 4 describes the execution of our proposed HGEMM algorithm, tiling design and pipelining optimizations in Ascend NPU environment in detail. Section 5 evaluates the effect of our proposed optimized HGEMM under a series of AI task workloads as well as benchmark workloads. Section 6 concludes the paper.

2. Motivation and Related Works

2.1. Motivation

Despite the variety of studies on low-bit matrix multiplications for efficient LLM inference [30–32], GPUs with Single-Instruction-Multiple-Thread (SIMT) instruction sets are the common hardware implementations in the research. Meanwhile, there is relatively less focus on low-precision GEMM based on NPUs with Single-Program-Multiple-Data (SPMD) instruction sets. More specifically, when it comes to Ascend NPU, HGEMM is the base of quantized lower-precision GEMMs due to hardware restrictions. In other words, weights stored in files in low precisions must be converted to FP16 precision before calling the GEMM operator on Ascend NPU. As a result, it is of great significance to utilize HGEMM on Ascend NPU with consideration of hardware characteristics, empowering HGEMM as well as quantized lower-precision GEMMs and further accelerated trainings on LLMs.

The Ascend NPU employs a separated layout design of Cube units and Vector units, which contrasts with the thread-centric execution of GPUs, presenting challenges in utilizing HGEMM that different optimization strategies for parallelism and instruction flow must be designed with. Although it is ideal to perform all HGEMM calculations in Cube units so as to fully exploit computability of Ascend NPU, necessary element-wise instructions can only be executed by Vector units, which is determined by hardware design, resulting in ineludible off-chip memory access and further affecting computation efficiency. Compounding this issue, prior work has shown that for most Machine Learning (ML) inference services, a single workload with varying input batch sizes can utilize less than half of the total available computational capacity, owing to the unbalanced occupancy of different compute units [33]. Thus, it is critical to design a streamlined instruction flow and corresponding optimization strategies to balance workload distribution between Cube units and Vector units, thereby enhancing overall computational efficiency.

2.2. Related Works

It is evident that the efficacy of GEMM optimization is heavily contingent on tiling strategies. Tiling essentially enhances memory access locality by partitioning matrix dimensions into smaller tiles that can be accommodated within different levels of the memory hierarchy. This, in turn, reduces the volume of high-overhead data transfers between global memory and computing units while boosting arithmetic intensity [34,35]. Hierarchical tiling designs, which are tailored to the concurrency levels of thread blocks, warps, and tensor cores, have emerged as a mainstream approach for GPU-based GEMM optimization. The HyTiS framework proposed by Zhang et al. [36] enables efficient GPU GEMM exe-

cution by integrating multi-level tiling with optimized wave scheduling, thus achieving substantial improvements in hardware resource utilization. Notably, this optimization methodology can be extended to the Ascend NPU architecture, provided that the Ascend platform is equipped with a hierarchical buffer design that is compatible with the core principles of multi-level tiling. Li et al. [24] present a coordinated tiling and batching framework that is applicable to both fixed-size and variable-size batched GEMM operations. The primary contribution of this work lies in its methodology for the quantized analysis of parallelism, which is realized by introducing a thread-level parallelism (TLP) model and a single-thread performance model. However, only effect on performance from BX and BY are explored while BK remains constant while seeking the best tiling strategy in this research since thread-centric GPU is the main focus on hardware environment. It is possible that reducing accumulation along the K direction may be of equal effectiveness on performance enhancement compared with increasing parallelism on Ascend NPU.

Apart from enhancing parallelism via advanced tiling designs, minimizing memory access-induced latency is equally critical. There are efforts from Ernst et al. [25], Rivera et al. [37], and Tang et al. [38] that focus on enhancement of memory access efficiency and cache hit rate. Park et al. [39] present LUT-GEMM that realizes quantized 4-bit precision matrix multiplication through a lookup table (LUT)-based computation technique, eliminating the resource-intensive dequantization process and reducing computational costs for weight-only quantization. NeuPIMs, a heterogeneous acceleration system raised by Heo et al. [40], jointly exploits GEMM-focused NPU and GEMV-optimized PIM devices, increasing efficiency in both compute-intensive GEMM and bandwidth-heavy GEMV operations throughout training of LLMs. In addition, there may exist multiple modes of memory access for certain hardware structure that behaves differently for varying data transfer efficiency for different granularity [22]. Optimizations targeting memory access efficiency play a pivotal role in boosting computational performance for tall-and-skinny matrices and batched small matrices, both of which are ubiquitous in LLM training workflows.

Furthermore, pipelining optimization—which overlaps data movement and computation stages to eliminate idle cycles—has emerged as a core research direction for enhancing GEMM efficiency, with extensive explorations conducted across heterogeneous hardware platforms. LiquidGEMM [41] presents an implicit fine-grained pipelining mechanism that enables full overlap of weight loading, dequantization, and matrix multiply-accumulate (MMA) operations among warp groups without software synchronization. This mechanism significantly hides the overhead of dequantization and improves hardware utilization. Another notable advancement is Stream-K++ [42], an enhanced scheduling algorithm for GPU GEMM that expands scheduling policies and integrates Bloom filters to rapidly eliminate unsuitable configurations. By optimizing pipeline workload balancing, it achieves substantial performance gains in specific scenarios. Regarding NPU architectures, pipelining optimization emphasizes adaptation to the characteristics of multi-core collaboration and heterogeneous integration. Taking the Ryzen AI NPU series as an example, systematic optimization methodologies spanning the XDNA and XDNA2 architectures enhance the pipelining of data transmission and computation by leveraging hardware-native features, thereby achieving state-of-the-art throughput for GEMM workloads in both INT8 and BF16 precision GEMM workloads [43].

3. Backgrounds

3.1. GEMM Routine

An $M \times N \times K$ GEMM performs a combined multiplication and accumulation on matrix: $C = \alpha AB + \beta C$, which involves two scalars: α and β , and three dense matrices: $A_{(M \times K)}$, $B_{(K \times N)}$, and $C_{(M \times N)}$, with $2MNK$ arithmetic operations in total. As presented

in Figure 1, when performing a GEMM, both matrices A and B will be first tiled into several panels. We specify $blockC_{(M_0 \times K_0)}$ as a single block in C , which is obtained from multiplication of $panelA_{(M_0 \times K)}$ and $panelB_{(K \times N_0)}$. Usually, the K dimension is so large that both $panelA$ and $panelB$ are further clipped along the K direction into several blocks so as to fit in on-chip buffers, and the result of $blockC$ becomes the following:

$$blockC = \sum_i panelA_i \times panelB_i \quad (1)$$

where $panelA_i$ stands for the i -th block in $panelA$, with a similar definition for $panelB$. Calculation of different $blockC$ can be assigned to different cores and executed simultaneously.

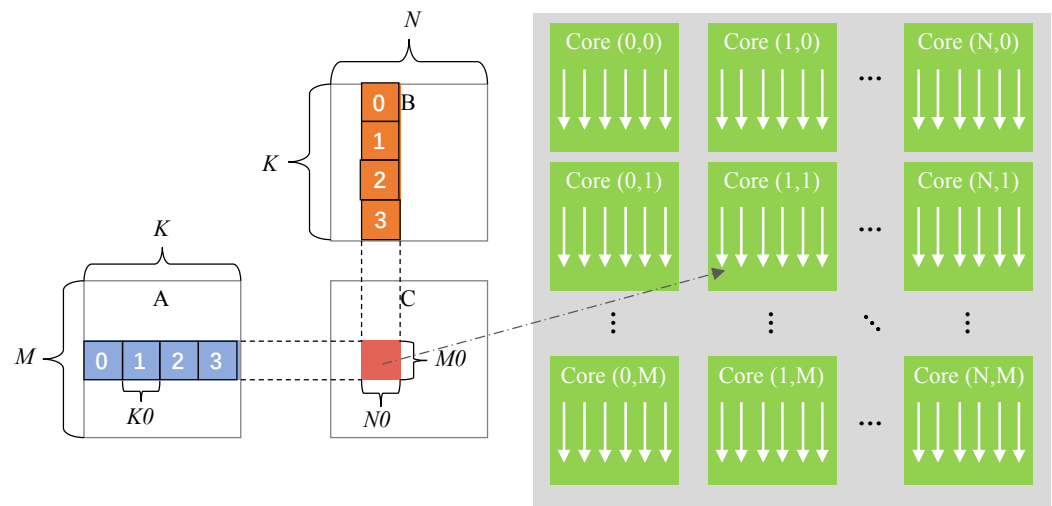


Figure 1. GEMM routine and its relationship with hardware structure.

3.2. Ascend NPU Architecture

The Ascend NPU is typically designed for versatile AI computations with outstanding power efficiency [29]. Unlike SIMT-based GPUs, SPMD instruction set is applied to Ascend NPU [44], where kernel function is launched to all AI Cores and execute simultaneously unless there is a defined synchronization instruction. Figure 2 shows the layout of Ascend NPU: each NPU consists of multiple AI Cores and they are connected with shared global memory (GM). Each AI Core contains one Cube unit, two Vector units and special function units. Specifically, the Cube unit is tailored for dense matrix–matrix multiplication operations. It accomplishes a $16 \times 16 \times 16$ matrix–matrix multiplication within a single clock cycle, corresponding to a total of 4096 arithmetic operations. The Vector unit, analogous to the thread unit in a GPU, processes 256-bit (i.e., 32-byte) element-wise operations per clock cycle. To enable parallel execution of Cube and Vector operations, dual-stream AIC/AIV pipelines are integrated into the design. In addition, the address of GM in the Ascend NPU adheres to a 512-byte alignment requirement, which is inherently dictated by its hardware design. Specifically, when the GM address satisfies this 512-byte alignment condition, data transfer efficiency is significantly enhanced. This alignment constraint, in turn, exerts a direct impact on both the design of the tiling strategy and the performance evaluation presented in subsequent sections.

The Memory Transfer Engine (MTE) manages the reading and writing of internal data between buffers at different levels, with a minimum granularity of 32 bytes. These buffers include the on-chip L1 buffer, the on-chip L0 buffer within the Cube unit, and the on-chip unified buffer (UB) associated with the Vector unit. As illustrated in Figure 3, the MTE comprises three operating modes, namely MTE1, MTE2, and MTE3. All MTE instructions

support programmable asynchronous data transfers, enabling parallel execution with the computational units.

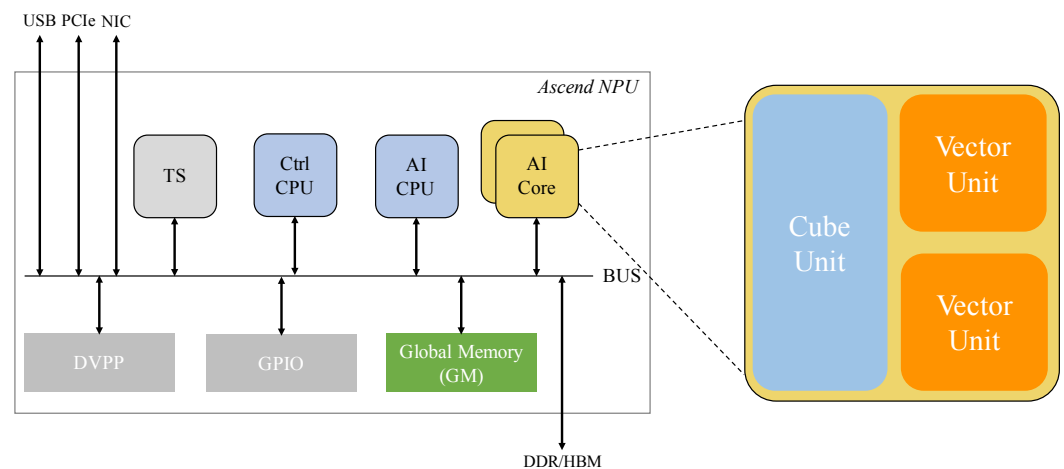


Figure 2. Architecture of Ascend NPUs [45].

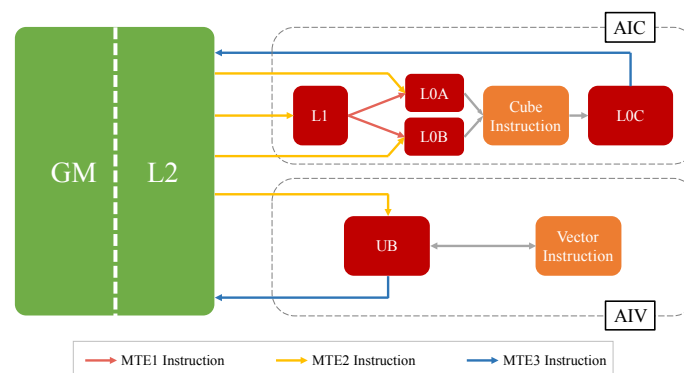


Figure 3. Dataflow within the AI Core under distinct MTE modes [46]. MTE instructions operating under different working modes support asynchronous execution.

4. Designing and Optimizing HGEMM on Ascend NPU

4.1. HGEMM at a Glance

Considering the aforementioned challenges and architectural characteristics, we propose a high-performance HGEMM implementation tailored for the Ascend NPU. This implementation coordinates Cube units and Vector units to achieve near-peak performance, comprising three core stages: preparation, matrix–matrix multiplication, and element-wise accumulation. To hide memory access latency, a double buffering strategy is deployed on the on-chip L0 buffers and UBs. Consequently, the maximum capacity of the L0 buffers for both input and output matrices is limited to (256×128) , or 32,768 elements in FP32 precision, which imposes constraints on the tiling design. Details of tiling design are further elaborated in Section 4.2. The design of our proposed GEMM is presented in Algorithm 1. Specifically, AIC and AIV instructions are dispatched to all AI Cores synchronously, with synchronization instructions integrated to regulate inter-core coordination.

To elaborate on the detailed design of our proposed HGEMM, we follow a sequential workflow as described below. First, we retrieve the global memory (GM) addresses of matrices A , B , and C , and load these matrices into the unified buffer (UB). Second, we perform padding on matrices A and B , and store the padded versions to specified GM addresses, while matrix C remains resident in the UB. Padding is well documented to significantly improve memory access efficiency [13,22] and is widely adopted in GEMM optimization [47,48]. In our algorithm, padding ensures that data adheres to the 512-byte

alignment requirement, thereby enhancing memory contiguity and boosting data transfer efficiency. Third, corresponding panels of matrices A and B are loaded into the L1 buffer, while single blocks of the $panelA$ and $panelB$ are further loaded into the L0A and L0B buffers, respectively. Starting from line 29, the Cube unit is responsible for executing the matrix–matrix multiplication operation $C_1 = AB$, with C_1 stored in L0C buffer in FP32 precision due to hardware architecture and numerical stability requirements, and quantized to FP16 precision upon being written back to GM. This step leverages the high-density computing capability of Cube unit, which facilitates the overall parallelism of the system. Fourth, beginning from line 14, the Vector unit performs the element-wise matrix–scalar multiplication $C_2 = \beta C$. Concurrently, matrix C_1 is loaded into the UB, and another element-wise matrix–scalar multiplication $C_3 = \alpha C_1$ is executed via a Vector unit starting from line 17. Finally, the Vector unit handles the element-wise accumulation operation $C = C_2 + C_3$, yielding a result that is fully consistent with the standard GEMM definition: $C = \alpha AB + \beta C$.

Algorithm 1 Code skeleton of HGEMM execution design on Ascend NPU

Input: Matrix A , B , and C in FP16 precision, Scalar α and β in FP16 precision.

Output: $C = \alpha AB + \beta C$ in FP16 precision.

CPU Instructions:

- 1: $panelnumA = \lceil M/M_0 \rceil, panelnumB = \lceil N/N_0 \rceil,$
 $panel2blocknum = \lceil K/K_0 \rceil, blocknumC = \lceil M/M_0 \rceil * \lceil N/N_0 \rceil$

AIV Instructions:

- 2: Reset all sync
 - 3: $UB_alpha \leftarrow MTE2(GM_alpha)$ //Load α and β from GM to UB
 - 4: $UB_beta \leftarrow MTE2(GM_beta)$
 - 5: **for** $i = 0$ to $panelnumA$ **do**
 - 6: Padding on $panelA[i]$
 - 7: **end for**
 - 8: **for** $i = 0$ to $panelnumB$ **do**
 - 9: Padding on $panelB[i]$
 - 10: **end for**
 - 11: Declare AIV padding complete and jump to line 21
 - 12: **for** $i = 0$ to $blocknumC$ **do**
 - 13: $UB_C \leftarrow MTE2(GM_C[i])$ //Load $blockC$ from GM to UB
 - 14: $C2 = UB_C \times UB_beta$
 - 15: Wait until AIC compute complete
 - 16: $UB_C1 \leftarrow MTE2(GM_C1[i])$
 - 17: $C3 = UB_C1 \times UB_alpha$
 - 18: $UB_result = C2 + C3$
 - 19: $GM_C[i] \leftarrow MTE3(UB_result)$
 - 20: **end for** //End of GEMM
 - AIC Instructions:**
 - 21: Wait until AIV padding complete
 - 22: **for** $i = 0$ to $blocknumC$ **do**
 - 23: Compute $colIdx$ and $rowIdx$
 - 24: $L1_A \leftarrow MTE2(panelA[rowIdx])$
 - 25: $L1_B \leftarrow MTE2(panelB[colIdx])$
 - 26: **for** $j = 0$ to $panel2blocknum$ **do**
 - 27: $L0_A \leftarrow MTE1(L1_A[j])$
 - 28: $L0_B \leftarrow MTE1(L1_B[j])$
 - 29: $L0_C += L0_A @ L0_B$ // $L0_C$ is FP32 precision
 - 30: **end for**
 - 31: $GM_C1[i] \leftarrow MTE3(L0_C)$ //Store C_1 to GM, convert FP32 to FP16
 - 32: Declare AIC compute complete and jump to line 15
 - 33: **end for**
-

4.2. Tiling Strategy Design in HGEMM

4.2.1. Parallelism Model and Single Core Model

This section demonstrates our tiling strategy in the proposed algorithm in detail. Tiling is critical to increasing parallelism and single thread performance on accelerating GEMMs. Usually, for a given matrix size, larger tile size decreases the parallelism since the number of tiles is decreased, while smaller tile size increases the parallelism and obtains better TLP. In work from Li et al. [24], a TLP model is introduced to quantize the parallelism in executing the GEMM. Considering batched GEMM scenarios where matrices are commonly batched along M or N directions, we define matrix C_i as the i -th single matrix which size is $(M_i \times N_i)$, and the corresponding tiling size is $(M_{0_i} \times N_{0_i})$. Since SPMD instruction sets are used on Ascend NPU, determining the number of AI Cores required in execution is more critical. With consideration of non-aligned cases, for a given tiling strategy, *aicore_num* which stands for amounts of AI Core, as well as *acc_loop* which stands for rounds of accumulations along the K direction in calculation of a single panel, are obtained as follows:

$$aicore_num = \sum_i (\lceil \frac{M_i}{M_{0_i}} \rceil \times \lceil \frac{N_i}{N_{0_i}} \rceil) \quad (2)$$

$$acc_loop = \max(\lceil \frac{K_i}{K_{0_i}} \rceil) \quad (3)$$

In this work, we only consider batch execution of fixed size matrix, which means M and N are constant. Effective batch execution of variable matrix size (e.g., MAGMA vbatched) is left for future works. Also, M_0 and N_0 are constant because there is only one best suitable tiling strategy assigned to each matrix size. Thus, *aicore_num* and *acc_loop* can be further simplified as follows:

$$aicore_num = \lceil \frac{M}{M_0} \rceil \times \lceil \frac{N}{N_0} \rceil \times n \quad (4)$$

$$acc_loop = \lceil \frac{K}{K_0} \rceil \quad (5)$$

where n stands for number of batches. We use $M \times N \times K = 256 \times 4096 \times 512$, $M_0 \times N_0 \times K_0 = 128 \times 512 \times 64$, and $n = 8$ as an example, $aicore_num = \lceil \frac{256}{128} \rceil \times \lceil \frac{4096}{512} \rceil \times 8 = 128$, indicating that 128 AI Cores in total will be participating in this example, and each AI Core will repeat accumulation $\lceil \frac{512}{64} \rceil = 8$ times in obtaining the result of a block of matrix C .

However, previous work mainly sets K_0 as a constant [24,49,50], which is likely results from GPU architecture. Moreover, in our proposed HGEMM, unlike thread-centric GPU that can perform multiplication and accumulation in a single SM, Vector units take charge of padding, matrix–scalar multiplication and element-wise accumulation operations, while Cube units take charge of matrix–matrix multiplication operations, indicating that data is transferred twice between Cube units and Vector units in calculating a single block of C , while data transfer in between is all conducted via off-chip GM. It can be inferred that there is little performance enhancement by only shrinking tile size since the times of accessing off-chip memory increases, despite the increase in *aicore_num* and parallelism. As a result, it is important to find a balance between increasing TLP and minimizing the need of data transfer between Cube units and Vector units.

4.2.2. Effects from Tiling Sizes Along M , N , and K Directions

Tiling Along M and N Directions

As mentioned, tiling size along M and N directions equally affects TLP, while tiling size along the K direction affects loops of accumulation, where instruction-level parallelism

(ILP) counts more. To determine the optimal tiling sizes for diverse matrix dimensions on Ascend NPUs, it is essential to separately investigate the performance impacts of tiling size variations along the M (N) and K directions. Although the selection of tiling sizes along these directions is not entirely independent in real-world deployments, we adopt a simplifying assumption in this performance characterization stage: M_0 (N_0) and K_0 are treated as independent variables. This assumption facilitates the clear identification of performance trends associated with changes in the tiling size of a single dimension, which is critical for informing the design of a robust self-adaptive tiling strategy algorithm in subsequent work.

We conclude the tiling strategy from previous studies [24,49,50] and apply the combination of M_0 and N_0 to Ascend NPU environment as Table 1, with all optimization strategies disabled temporarily. For single square matrix cases, we adopt different tiling strategies listed in Table 1 for testing, with K_0 fixed at 8. This is conducted to keep in line with the configurations used in previous studies. As shown in Figure 4, the horizontal axis shows the GFlops of single GEMM, and the vertical axis shows different matrix sizes where $M = N = K$. It reveals that for single square matrix cases where M , N , and K are greater than 256, small tiling size causes major drawbacks on performance, despite the increase in the number of AI Cores involved leading to better parallelism in theory. This is due to decline on memory access efficiency with the decrease in data size transferring between buffers. In addition, accesses to the GM cannot be executed asynchronously as long as addresses in GM fall within a consecutive 512-byte range, which is determined by hardware design, resulting in performance fluctuation.

Table 1. Tiling Size for M_0 and N_0 .

<i>index_mn</i>	M_0	N_0
0	16	16
1	32	32
2	64	64
3	128	128
4	256	128

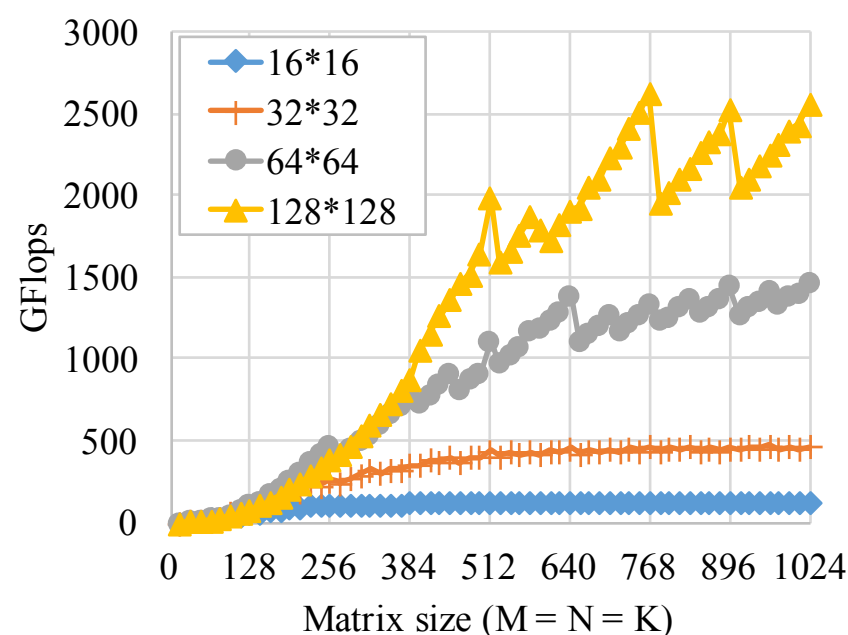


Figure 4. HGEEM performance under different M_0 and N_0 ($K_0 = 8$).

It can also be discovered that for matrices where M , N , and K are smaller than 256, there is no significant influence on performance under different tiling sizes. This is because when executing such GEMM, data input and output is so small that there is always some AI Core left idle regardless of tiling sizes. Considering that the effect on performance from tiling size varies significantly between M , N , and K , more than 256 and no more than 256, we define the former one as general matrix, and the latter as small matrix.

Tiling Along K Direction

Previous research focus on increasing TLP by adjusting M_0 and N_0 , while there is relatively little focus on effect from K_0 . However, considering the major difference in instruction sets as well as hardware architecture, it is also critical to investigate the effect on performance from tiling along the K direction under Ascend NPU environment, which determines the loops in accumulation. It is likely that performance utilization can also be obtained from reduced times of accumulation.

To work out, we fix $M_0 = N_0 = 32$ and apply different K_0 from 8 to 512, respectively, for single square matrices. This ensures block sizes of matrices can be fully accommodated within buffers. All optimization strategies are also disabled temporarily in this stage. As of Figure 5, it is clear that there is significant performance enhance with reduced loops along the K direction by increasing K_0 . This is critical to the design of tiling size selection strategy which is explained in the following section.

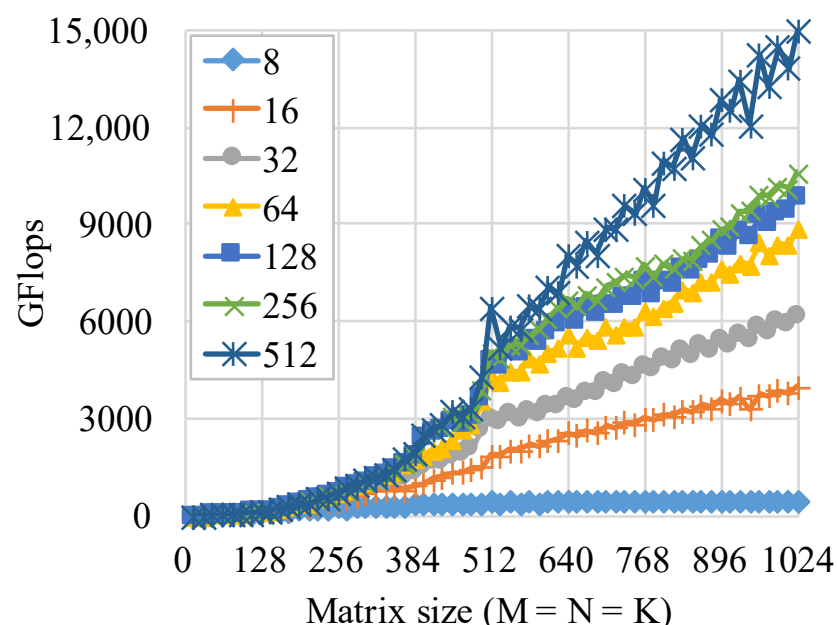


Figure 5. HGEMM performance under different K_0 ($M_0 = N_0 = 32$).

4.2.3. Tiling Strategy Algorithm Design

When designing tiling strategy, matters that should be considered are as follows: (i) the maximum output of matrix size for each AI Core is (256×128) , or 32,768 elements in FP32 precision, and (ii) apart from determining suitable M_0 and N_0 , selection of K_0 is also critical so as to reduce loops of accumulation and obtain better performance.

We design a series of K_0 as shown in Table 2. To obtain the best tiling strategy, we first consider larger K_0 so as to reduce the loops of accumulation, which significantly contributes to performance enhancement. However, when M and N are also large, increasing M_0 and N_0 by sacrificing K_0 to attempt to lower excessive number of AI Cores becomes more critical. The target of the tiling strategy is to strike a balance between lowering loops of accumulation and lowering the number of AI Cores involved if they exceed the threshold.

The algorithm uses two queues, namely queue A for K_0 and queue B for combination of M_0 and N_0 . It consists of the following steps:

1. Determine K_0 selectable as listed in Table 2, depending on input of K , which should satisfy $K_0 < 2K$, and put all available K_0 into priority queue A where larger K_0 is given higher priority.
2. Pop K_0 from priority queue A .
3. Determine M_0 and N_0 selectable as listed in Table 1, where the corresponding *index_mn* must not exceed *index_mn_max* which is obtained from popped K_0 , and put all available combination of M_0 and N_0 into priority queue B where smaller M_0 and N_0 combination is given higher priority. This is to ensure that each block for both input and output matrices does not overflow from L0A, L0B, and L0C buffers.
4. Pop one combination of M_0 and N_0 from queue B and calculate *aicore_num* from Equation (4).
5. Compare *aicore_num* with the threshold. If *aicore_num* is smaller than the threshold, current M_0 , N_0 , and K_0 are selected as the final solution of the tiling size. If not, then (i) if corresponding combination is not the last element in queue B , pop the next combination in queue B and return to step 4, and (ii) if this is already the last element in queue B , the next K_0 in queue A is popped, and one must return to step 3. Furthermore, if the corresponding K_0 is already the last element in queue A , we will select current M_0 , N_0 , and K_0 as the final solution, which should satisfy $M_0 \times N_0 \times K_0 = 256 \times 128 \times 128$.

Table 2. Tiling Size for K_0 .

<i>index_k</i>	K_0	<i>index_mn_max</i>
0	128	4
1	256	3
2	512	2
3	1024	1

We use the example introduced in Section 1 where the matrix size is $2048 \times 128 \times 2048$ and $n = 1$ to demonstrate our proposed tiling strategy algorithm. K_0 selectable are 128, 256, 512, and 1024 as of Table 2. First, all selectable K_0 are put into priority queue A . Then, we pop $K_0 = 1024$ from queue A and the corresponding *index_mn_max* is 1, which indicates the first and second combination of M_0 and N_0 as Table 1 can be put into priority queue B . Next, *index_mn* = 0, where $M_0 = N_0 = 16$ is popped. *aicore_num* is 1024. For Atlas 300T A2 hardware configuration, we set the threshold to 20, and *aicore_num* is greater than the threshold; thus, we return to step 3 and pop *index_mn* = 1 where $M_0 = N_0 = 32$, and *aicore_num* becomes 256, which is still greater than the threshold. Since this is the last element in queue B , indicating none of the tiling strategy for M_0 and N_0 satisfies the threshold under this K_0 , we pop the next element in queue A where $K_0 = 512$ with corresponding *index_mn_max* as 2 and return to step 2. By repeating several times, it is found that when *index_mn* = 3, the corresponding tiling size is $M_0 = N_0 = 128$, and *aicore_num* becomes 16 which satisfies the threshold. Thus, the final tiling size of this GEMM input is $128 \times 128 \times 256$.

To validate the global optimality of the proposed algorithm, we select three representative GEMM scenarios: regular matrices, tall-and-skinny matrices with dominant M dimension, and tall-and-skinny matrices with dominant K dimension, all with identical total arithmetic operations. All optimization strategies are disabled temporarily in this validation stage. As illustrated in Figure 6, the tiling combination generated by the tiling strategy algorithm coincides with that corresponding to the maximum performance

across all representative GEMM scenarios, thus verifying the global optimality of the tiling strategy algorithm.

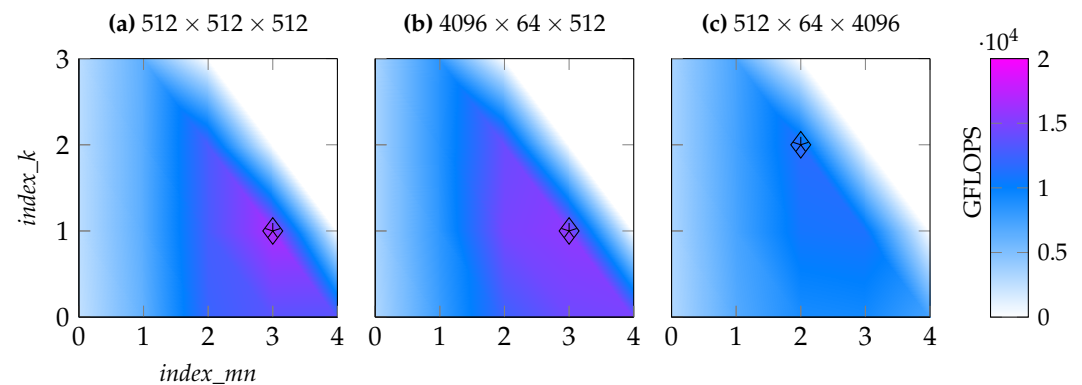


Figure 6. HGEMM performance over the joint tiling search space under typical scenarios: the tiling combination corresponding to the maximum performance is represented by a diamond, and the result derived from the tiling strategy algorithm by a star. Matrix dimensions are denoted as $M \times N \times K$.

4.3. Pipelining Optimizations on HGEMM

Since the optimal tiling strategy and tiling size selection criteria tailored for the Ascend NPU have been designed and implemented, this section focuses on the block-wise pipelining optimization as well as the hardware occupancy utilization of the proposed HGEMM. In our proposed GEMM, the sequence of major steps is as follows: (i) padding on matrix A and B on Vector units, (ii) matrix–matrix multiplication $C_1 = AB$ on Cube units, (iii) matrix–scalar multiplications $C_2 = \beta C_1$, $C_3 = \alpha AB$ on Vector units, and (iv) element-wise accumulation $C = C_2 + C_3$ on Vector units. However, for cases where the matrix is relatively small, the hardware computational capacity cannot be fully harnessed—and this inadequacy is specifically reflected in the inevitable idling of AI Cores. Additionally, the bandwidth capability of the GM is expected to become a significant bottleneck limiting computational efficiency, unless sophisticatedly orchestrated pipelining is employed.

To address these challenges, a series of optimization strategies including block-wise pipelining, *ShuffleK* and *SplitK*, have been designed and applied, where the strategies either enhance memory access efficiency or improve AI Core occupancy. The following sections elaborate on the design principles and hardware adaptation mechanisms of each strategy:

4.3.1. Block-Wise Pipelining Optimizations on HGEMM

As depicted in Figure 7, the HGEMM pipeline for single-block computation orchestrates a fused sequence of operations that seamlessly integrates MTE operations and Vector-units-based element-wise calculations with Cube-units-based matrix–matrix multiplications, capitalizing on the dual-stream architecture of Ascend NPU to enable concurrent execution. This approach alleviates bandwidth limitations by reducing accesses to GM and enhancing on-chip data reuse in the UB as well as L1 and L0 buffers.

The procedure initiates with padding on the Vector units, before transferring padded *panelA* and *panelB* from GM to L1 buffer, with operations of prefetching next *panelA* and *panelB* from GM to double-buffered L1 buffer executed if available. The prefetch operation ensures consecutive MTE2-mode data transfer of *panelA* and *panelB* from GM, utilizing memory bandwidth ability. It is also obvious that consecutive matrix–matrix multiplication calculation is achieved through sophisticated pipelining, maximizing hardware computability. These operations are synchronized through hardware events to support pipelined processing, with double-buffering strategy, which alternates read/write between two parts of buffers, mitigating stalls.

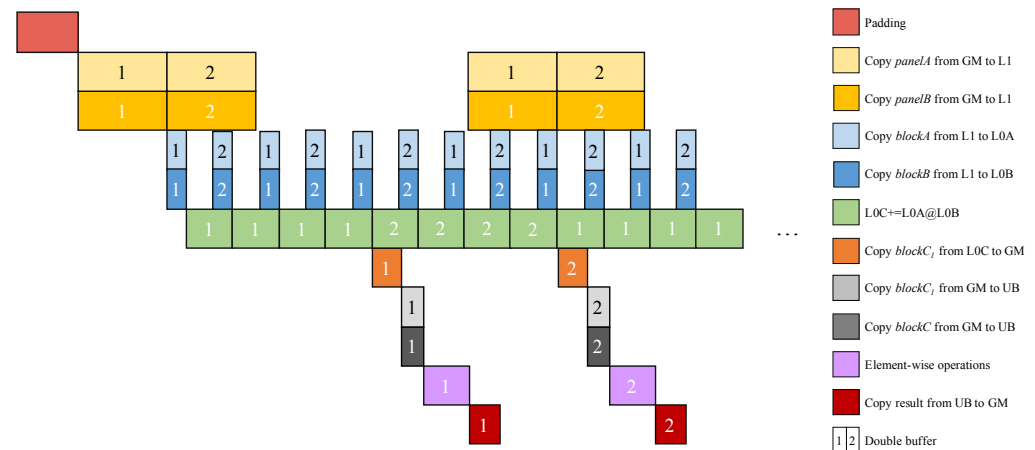


Figure 7. Pipelining of the proposed HGEMM with double buffer and prefetch strategies.

4.3.2. *ShuffleK*: Eliminating Memory Access Conflicts in HGEMM

As previously mentioned, the bandwidth capability of the GM is expected to become a significant bottleneck limiting computational efficiency in HGEMM. To make things worse, determined by the hardware architecture, accesses to the GM cannot be executed asynchronously and must be performed serially, provided that all these addresses fall within a consecutive 512-byte range, even if their addresses are different from each other, causing significant drawback on memory access efficiency.

The *ShuffleK* method is proposed to mitigate the issue of potential under-utilization of memory access efficiency. As an example, illustrated in Figure 8a, without implementing *ShuffleK*, multiple AI Cores are required to access to the same address to *panelA* in order to obtain the desired intermediate results, where asynchronous access is not supported by hardware architecture, causing low access efficiency. On the contrary, as presented in Figure 8b, the *ShuffleK* method reorders the distribution sequence of sub-tasks along the *K* direction in computing different blocks of matrix *C* so that different addresses to *panelA* can be accessed in parallel, breaking the uniform serial sub-task distribution pattern across AI Cores, thereby mitigating the occurrence of serial access to the GM and further improving overall memory access efficiency.

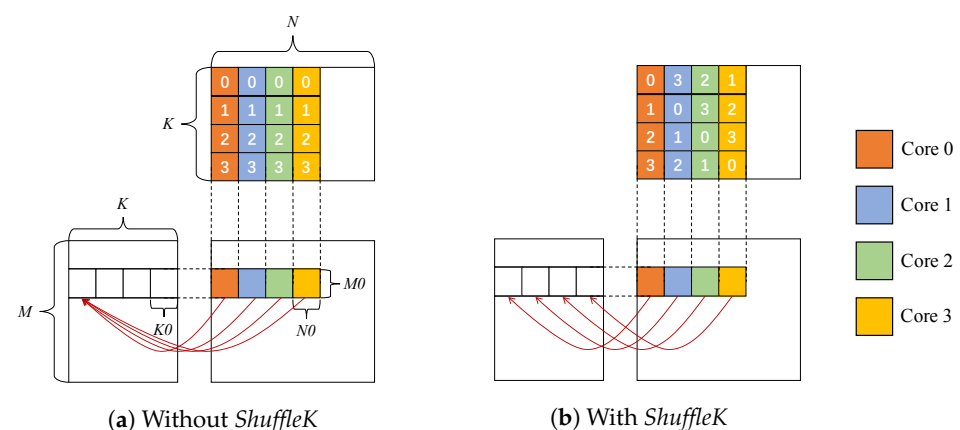


Figure 8. *ShuffleK* mechanism principle and memory access differences with and without *ShuffleK* implementation. The mechanism breaks the uniform serial sub-task distribution pattern and mitigates the occurrence of serial access to the GM.

4.3.3. *SplitK*: Utilizing AI Core Occupancy for Small Matrix Scenarios

Although the best tiling strategy for each matrix case is realized through a self-adaptive algorithm, the total computational load may still be insufficient to fully occupy all AI Cores

under small matrix scenarios, resulting in under-utilization of hardware resources that directly reduces the overall computational efficiency of HGEMM.

To deal with insufficient workload, the *SplitK* method is raised. Specifically, the *SplitK* strategy mitigates the issue of computational under-utilization by partitioning the matrix multiplication task along the K dimension and distributing the sub-tasks across multiple AI Cores. This distribution is implemented to compute a single block of matrix C , thereby enhancing parallelism during the matrix multiplication phase.

As depicted in Figure 9, the split sub-tasks are distributed to individual AI Cores for parallel computation. Each AI Core independently executes the full pipeline of operations (padding, Cube-based multiplication, Vector-based scalar operation, and accumulation) on its assigned sub-task. After all sub-tasks are completed, the intermediate results (i.e. sub-matrices of C) are element-wise accumulated to form the final output matrix C , with atomic instructions implemented. These atomic instructions ensure mutually exclusive access to shared memory regions where intermediate sub-matrices are stored, eliminating race conditions and guaranteeing the correctness of the aggregated final result while minimizing synchronization overhead.

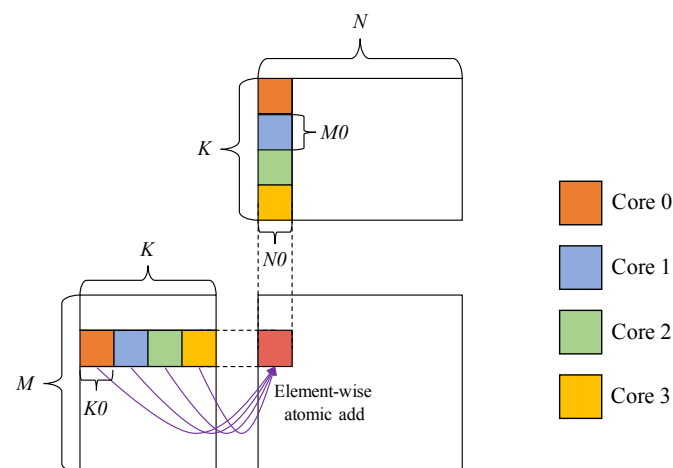


Figure 9. Principle of the *SplitK* method. The split sub-tasks are distributed to individual AI Cores for parallel computation, with atomic instructions implemented to eliminate race conditions and guarantee the correctness of the aggregated result.

5. Evaluation

5.1. Experimental Setup

In this section, the proposed HGEMM is evaluated. Huawei Atlas 300T A2 Training Card [51] designed by Huawei Technologies Co., Ltd., Shenzhen, China, is selected as the Ascend NPU environment. *GemmExample* based on CATLASS template [10] running on Huawei Atlas 300T A2 Training Card, as well as HGEMM from cuBLAS library running on Nvidia A800 GPU [52] are selected for comparison. Table 3 presents a more detailed description on hardware specifications as well as implementation details.

Given the popularity and significance of HGEMM in a variety of AI tasks as well as benchmarks, our proposed HGEMM is evaluated under (i) general random workloads, (ii) benchmark-based workloads, and (iii) open-pre-trained-Transformers-based (OPT-based) workloads [53]. Values in elements in matrices A , B , and C as well as constant α , β are randomly generated between -1 and 1 in FP16 precision. All optimizations are enabled throughout the evaluation session. For each run, HGEMM execution is repeated 25 times where the first 20 times acts as warmup, and the average execution time of the last 5 times is taken as the result. With consideration that a single HGEMM accomplishes

2MNK arithmetic operations in total, the performance metric of TFLOPS is calculated as the following equation:

$$\text{TFLOPS} = \frac{2 \times M \times N \times K}{\text{totaltime} \times 10^{12}} \quad (6)$$

where M , N , and K represent the dimensions of the matrices, and *totaltime* is the total execution time (in seconds) of a single run of HGEMM.

Table 3. Implementation details and hardware specifications for HGEMM performance evaluation.

	Huawei Atlas 300T A2	Nvidia A800
Buffers	L1 512 KB (per core)	L1 192 KB (per SM)
Caches	L2 192 MB	L2 40 MB
Memory	64 GB HBM2e	80 GB HBM2e
Memory Bandwidth	1.6 TB/s	2.04 TB/s
Theoretical FP16 Computability	280 TFLOPS	312 TFLOPS ¹
Programming languages	Ascend C	C++
Compilers	Bisheng and gcc	NVCC
Software version	CANN 8.0.0	cuBLAS 12.6
GEMM implementation	(1) Our proposed HGEMM (2) GemmExample	cublasHgemm

¹ With Tensor Core implementation.

5.2. HGEMM Performance on General Random Workloads

For general random workloads, 5000 synthetic GEMM tasks are generated, with matrix dimensions M , N , and K randomly sampled between 256 and 10,000, in order to assess robustness across varied matrix shapes. As listed in Table 4, our proposed HGEMM attains an average performance of 182.62 TFLOPS, achieving an average $3.56\times$ speedup over GemmExample (based on CATLASS templates) under identical hardware configurations and an average $2.10\times$ speedup over the cuBLAS implementation on the Nvidia A800 GPU. Additionally, as illustrated in Figure 10, significant performance fluctuation can be discovered on Nvidia GPU, which likely results from alignment issues. It further reveals that the padding operation integrated into the proposed HGEMM plays a critical role in enabling highly efficient execution across diverse matrix shapes and mitigating performance fluctuation, especially for scenarios where initial 512-byte alignment is not satisfied.

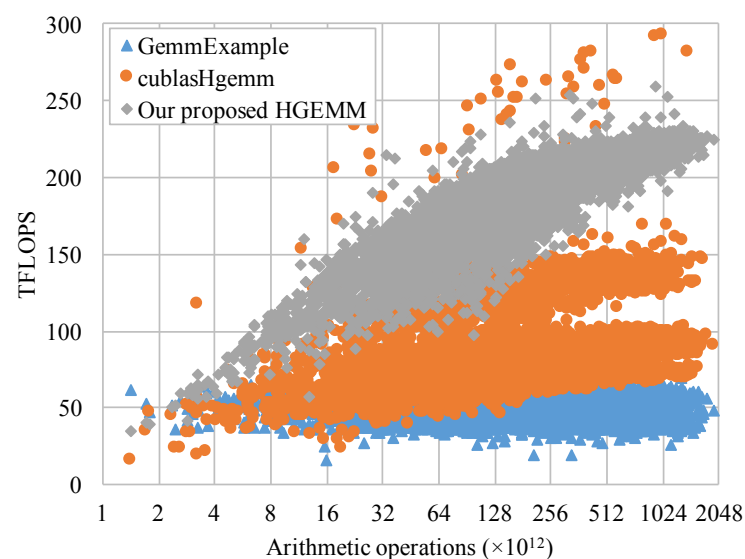


Figure 10. Performance comparison of the proposed HGEMM under different general random workloads.

Table 4. HGEMM performance comparison under general random workloads.

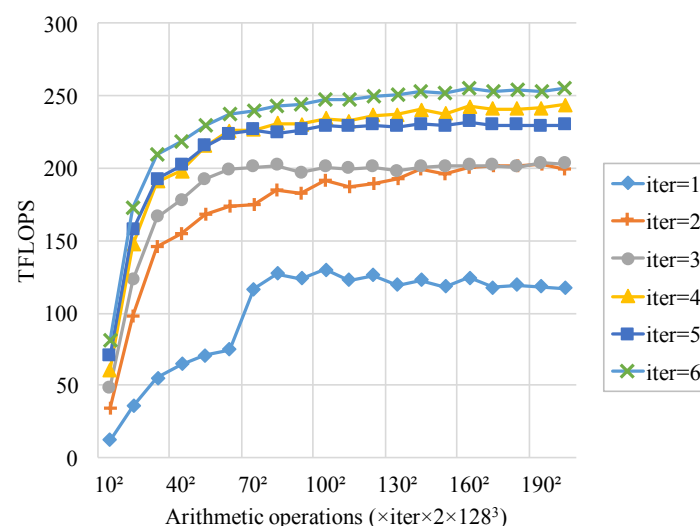
Kernel	Hardware	Maximum TFLOPS	Average TFLOPS	Average Speedup
GemmExample	Huawei Atlas 300T A2	69.10	51.25	1.0×
cublasHgemm	Nvidia A800	292.89	86.86	1.7×
Our proposed HGEMM	Huawei Atlas 300T A2	259.19	182.62	3.6×

5.3. HGEMM Performance on HPL-MxP-Based Benchmark Workloads

HPL-MxP is a mixed-precision benchmark which employs both a lower-precision LU factorization with a non-stationary iterative refinement based on generalized minimal residual method (GMRES), providing a unified framework for testing hardware capabilities relevant to both high-performance computing (HPC) and AI workloads [54].

With consideration of the implementation of HGEMM in HPL-MxP on Ascend NPU, we set M and N to vary between 128 and 25,600, while K varies between 128 and 768, with M , N , and K stick proportional to 128, in order to keep in line with the demand on trailing matrix update as well as multi-iteration-fused matrix update under the specified hardware environment [55]. It is worth pointing out that the trailing matrix update is the last stage in LU factorization; thus, the preparation stage of the HGEMM can be skipped so that only the matrix–matrix multiplication stage and element-wise accumulation stage are involved under HPL-MxP-based benchmark workloads.

Due to the fact that HGEMM from cuBLAS library does not support kernel-level modifications by programmers, we only evaluate our proposed HGEMM under Ascend NPU environment in benchmark scenarios. As presented in Figure 11, high efficiency HGEMM is achieved as long as binding multiple iterations of trailing matrix update into single GEMM, with highest TFLOPS of 255.57, accounting for a utilization ratio of over 90% when binding 6 rounds of update in a single run, and 187.35 TFLOPS on average.

**Figure 11.** Performance of the proposed HGEMM under different benchmark workloads on Ascend NPU.

5.4. HGEMM Performance on OPT-Inspired Workloads

It has already been a common sense that LLMs have shown remarkable capabilities for zero- and few-shot learning. The OPT family models are a suite of decoder-only pre-trained transformers, with a scale of parameters ranging from 125 M to 175 B [53]. For targeted evaluation, we focus on FFN-like shapes from OPT models, which satisfies hidden size

$K = 7168$ and intermediate size $N = 28,672$ with batch sizes M that vary between 8 and 512, and align with practical batched inference in production LLM deployments.

Table 5 summarizes HGEMM performance under scaled OPT-based workloads. Evidently, our proposed HGEMM significantly outperforms GemmExample under identical hardware configurations, achieving an average $4.40\times$ speedup. Additionally, while the cuBLAS implementation on Nvidia GPUs exhibits superior performance at small batch sizes, our proposed HGEMM on Ascend NPUs consistently outperforms cublasHgemm as batch sizes increase, and delivers comparable efficiency in bandwidth-limited LLM inference, with gains of up to $1.04\times$ when $M = 256$.

Table 5. HGEMM performance comparison under scaled OPT-based workloads ($K = 7168$, $N = 28,672$). The durations are reported in microseconds (us).

M	cublasHGEMM		GemmExample		Our Proposed HGEMM	
	TFLOPS	Duration	TFLOPS	Duration	TFLOPS	Duration
8	12.48	263.55	2.54	1292.42	5.97	550.90
16	25.47	258.18	4.93	1333.13	12.86	511.33
32	49.16	267.58	9.28	1417.23	26.13	503.34
64	88.81	296.22	19.73	1333.62	52.37	502.37
128	143.85	365.76	35.33	1489.37	104.72	502.40
256	190.37	552.74	36.89	2852.70	197.55	532.67
512	243.56	864.06	37.47	5617.02	244.00	862.50

6. Conclusions

As one of the most widely used high performance kernels, GEMM plays a cornerstone role in a variety of fields of AI applications. It is of great significance to design and implement high-efficiency low-precision GEMM on modern NPU environment as the trainings of CNNs and LLMs are gaining increasing popularity. In this work, HGEMM for Ascend NPU is presented that realizes cooperation between Cube units and Vector units to handle matrix–matrix multiplications and element-wise calculations, respectively. To obtain better parallelism, a self-adaptive tiling mechanism is developed, where impacts from M , N , and K directions are comprehensively considered. In addition, a series of optimization strategies including block-wise pipelining, *ShuffleK* and *SplitK*, are designed and implemented that eliminates low-efficiency memory access, further utilizing HGEMM performance on Ascend NPU environment. Evaluation results demonstrate that the proposed HGEMM achieves an average $3.56\times$ speedup over GemmExample based on CATLASS templates under identical Ascend NPU configurations, $2.10\times$ speedup relative to the cuBLAS implementation on the Nvidia A800 GPU under general random workloads; it also achieves a maximum computational utilization exceeding 90% under benchmark workloads. Moreover, while the proposed HGEMM significantly outperforms GemmExample, it delivers efficiency comparable to the cuBLAS implementation in OPT-based bandwidth-limited LLM inference workloads.

Possible future work lies in applying our proposed GEMM under more general workloads, including variable batch sizes, different data layout patterns, as well as different memory access modes. In addition, HGEMM in this study is based on a single NPU execution condition. For the condition of multiple NPU execution, optimizing broadcast strategy, memory access mode, and overlap between computation and transmission also becomes critical, which may lead to a completely different realization of GEMM. The tiling strategy selection criteria is also inspiring in optimizing GEMM under different NPU hardware environment and can be further investigated in future study.

Author Contributions: Conceptualization, E.Z. and L.L.; methodology, E.Z.; validation, E.Z., P.X. and L.L.; formal analysis, E.Z.; writing—original draft preparation, E.Z.; writing—review and editing,

P.X. and L.L.; visualization, E.Z.; supervision, L.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Cicek, N.M.; Shen, X.; Ozturk, O. Energy Efficient Boosting of GEMM Accelerators for DNN via Reuse. *ACM Trans. Des. Autom. Electron. Syst.* **2022**, *27*, 1–26. <https://doi.org/10.1145/3503469>.
2. Drmač, Z. A LAPACK Implementation of the Dynamic Mode Decomposition. *ACM Trans. Math. Softw.* **2024**, *50*, 1–32. <https://doi.org/10.1145/3640012>.
3. Nair, H.; Vellaisamy, P.; Chen, A.; Finn, J.; Li, A.; Trivedi, M.; Shen, J.P. tuGEMM: Area-Power-Efficient Temporal Unary GEMM Architecture for Low-Precision Edge AI. In Proceedings of the 2023 IEEE International Symposium on Circuits and Systems (ISCAS), Monterey, CA, USA, 21–25 May 2023; pp. 1–5. <https://doi.org/10.1109/ISCAS46773.2023.10181357>.
4. NVIDIA. cuBLAS (v12.5). Available online: <https://docs.nvidia.com/cuda/archive/12.5.0> (accessed on 21 May 2024).
5. Advanced Micro Devices, Inc. rocBLAS 4.1.2 Documentation. Available online: <https://rocm.docs.amd.com/projects/rocBLAS/en/docs-6.1.2/index.html> (accessed on 4 June 2024).
6. Xu, R.G.; Van Zee, F.G.; van de Geijn, R.A. Towards a Unified Implementation of GEMM in BLIS. In Proceedings of the 37th ACM International Conference on Supercomputing, ICS '23, Orlando, FL, USA, 21–23 June 2023; pp. 111–121. <https://doi.org/10.1145/3577193.3593707>.
7. Abdelfattah, A.; Haidar, A.; Tomov, S.; Dongarra, J. Novel HPC techniques to batch execution of many variable size BLAS computations on GPUs. In Proceedings of the International Conference on Supercomputing, ICS '17, Chicago, IL, USA, 13–16 November 2017. <https://doi.org/10.1145/3079079.3079103>.
8. Nvidia. CUDA Templates for Linear Algebra Subroutines. Available online: <https://github.com/NVIDIA/cutlass> (accessed on 29 October 2025).
9. Kerr, A.; Merrill, D.; Demouth, J.; Tran, J. CUTLASS: Fast Linear Algebra in CUDA C++. *Nvidia*, 5 December 2017. Available online: <https://developer.nvidia.com/blog/cutlass-linear-algebra-cuda/> (accessed on 6 January 2026).
10. Huawei. Catlass: CANN Templates for Linear Algebra Subroutines. Available online: <https://gitcode.com/cann/catlass> (accessed on 29 October 2025).
11. Chen, Y.; Lu, L. AscQLUT: A Decode-Fused INT4 GEMM Kernel for Accelerating Low-Bit Quantized Matrix Multiplication via Lookup Tables on Ascend 910B NPU. **2025**. <https://doi.org/10.2139/ssrn.5679592>.
12. Ma, Z.; Wang, H.; Feng, G.; Zhang, C.; Xie, L.; He, J.; Chen, S.; Zhai, J. Efficiently emulating high-bitwidth computation with low-bitwidth hardware. In Proceedings of the 36th ACM International Conference on Supercomputing, ICS '22, Virtual Event, 28–30 June 2022. <https://doi.org/10.1145/3524059.3532377>.
13. Hong, K.; Dai, G.; Xu, J.; Mao, Q.; Li, X.; Liu, J.; Chen, K.; Dong, Y.; Wang, Y. FlashDecoding++: Faster Large Language Model Inference with Asynchronization, Flat GEMM Optimization, and Heuristics. *Mach. Learn. Syst.* **2024**, *6*, 148–161.
14. Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv* **2023**, arXiv:2307.09288.
15. Dettmers, T.; Lewis, M.; Belkada, Y.; Zettlemoyer, L. GPT3.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 30318–30332.
16. Xia, H.; Zheng, Z.; Wu, X.; Chen, S.; Yao, Z.; Youn, S.; et al. Quant-LLM: Accelerating the Serving of Large Language Models via FP6-Centric Algorithm-System Co-Design on Modern GPUs. In Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC 24), Santa Clara, CA, USA, 10–12 July 2024; pp. 699–713.
17. Xi, H.; Li, C.; Chen, J.; Zhu, J. Training Transformers with 4-bit Integers. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 49146–49168.
18. Wu, X.; Li, C.; Yazdani Aminabadi, R.; Yao, Z.; He, Y. Understanding Int4 Quantization for Language Models: Latency Speedup, Composability, and Failure Cases. In Proceedings of the 40th International Conference on Machine Learning, PMLR, Honolulu, HI, USA 23–29 July 2023; Volume 202, pp. 37524–37539.
19. Abdelfattah, A.; Haidar, A.; Tomov, S.; Dongarra, J. Performance, Design, and Autotuning of Batched GEMM for GPUs. In *High Performance Computing*; Kunkel, J.M., Balaji, P., Dongarra, J., Eds.; Springer: Cham, Switzerland, 2016; pp. 21–38.
20. Dongarra, J.; Hammarling, S.; Higham, N.J.; Relton, S.D.; Valero-Lara, P.; Zounon, M. The Design and Performance of Batched BLAS on Modern High-Performance Computing Systems. *Procedia Comput. Sci.* **2017**, *108*, 495–504. <https://doi.org/10.1016/j.procs.2017.05.138>.

21. Abdelfattah, A.; Costa, T.; Dongarra, J.; Gates, M.; Haidar, A.; Hammarling, S.; Higham, N.J.; Kurzak, J.; Luszczek, P.; Tomov, S.; et al. A Set of Batched Basic Linear Algebra Subprograms and LAPACK Routines. *ACM Trans. Math. Softw.* **2021**, *47*, 1–23. <https://doi.org/10.1145/3431921>.
22. Jiang, L.; Yang, C.; Ma, W. Enabling Highly Efficient Batched Matrix Multiplications on SW26010 Many-core Processor. *ACM Trans. Archit. Code Optim.* **2020**, *17*, 1–23. <https://doi.org/10.1145/3378176>.
23. Mijić, N.; Davidović, D. Batched matrix operations on distributed GPUs with application in theoretical physics. In Proceedings of the 2022 45th Jubilee International Convention on Information, Communication and Electronic Technology (MIPRO), Opatija, Croatia, 23–27 May 2022; pp. 293–299. <https://doi.org/10.23919/MIPRO55190.2022.9803591>.
24. Li, X.; Liang, Y.; Yan, S.; Jia, L.; Li, Y. A coordinated tiling and batching framework for efficient GEMM on GPUs. In Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming, PPoPP '19, Washington, DC, USA, 16–20 February 2019; pp. 229–241. <https://doi.org/10.1145/3293883.3295734>.
25. Ernst, D.; Hager, G.; Thies, J.; Wellein, G. Performance engineering for real and complex tall & skinny matrix multiplication kernels on GPUs. *Int. J. High Perform. Comput. Appl.* **2021**, *35*, 5–19. <https://doi.org/10.1177/1094342020965661>.
26. Amrouch, H.; Zervakis, G.; Salamin, S.; Kattan, H.; Anagnostopoulos, I.; Henkel, J. NPU Thermal Management. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **2020**, *39*, 3842–3855. <https://doi.org/10.1109/TCAD.2020.3012753>.
27. Georgie, P. *What Is an NPU? Here's Why Everyone's Suddenly Talking About Them*; Digital Trends Media Group: Portland, OR, USA, 27 December 2023.
28. Lee, K.J. Chapter Seven - Architecture of neural processing unit for deep neural networks. In *Hardware Accelerator Systems for Artificial Intelligence and Machine Learning*; Kim, S., Deka, G.C., Eds.; Elsevier: Amsterdam, The Netherlands, 2021; Volume 122, Advances in Computers, pp. 217–245. <https://doi.org/10.1016/bs.adcom.2020.11.001>.
29. Liao, H.; Tu, J.; Xia, J.; Liu, H.; Zhou, X.; Yuan, H.; et al. Ascend: A Scalable and Unified Architecture for Ubiquitous Deep Neural Network Computing: Industry Track Paper. In Proceedings of the 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA), Virtually, 27 February–3 March 2021; pp. 789–801. <https://doi.org/10.1109/HPCA51647.2021.00071>.
30. Hoffmann, J.; Borgeaud, S.; Mensch, A.; Buchatskaya, E.; Cai, T.; Rutherford, E.; de Las Casas, D.; Hendricks, L.A.; Welbl, J.; Clark, A.; et al. Training Compute-Optimal Large Language Models. *arXiv* **2022**, arXiv:2203.15556.
31. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, L.u.; Polosukhin, I. Attention is All you Need. In *Advances in Neural Information Processing Systems*; Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R., Eds.; Curran Associates, Inc.: Red Hook, NY, USA, 2017; Volume 30.
32. Guo, H.; Guo, N.; Meinel, C.; Yang, H. Low-bit CUTLASS GEMM Template Auto-tuning using Neural Network. In Proceedings of the 2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA), Kaifeng, China, 30 October–2 November 2024; pp. 394–401. <https://doi.org/10.1109/ISPA63168.2024.00057>.
33. Xue, Y.; Liu, Y.; Nai, L.; Huang, J. V10: Hardware-Assisted NPU Multi-tenancy for Improved Resource Utilization and Fairness. In Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA '23, Orlando, FL, USA 17–21 June 2023. <https://doi.org/10.1145/3579371.3589059>.
34. Wang, C.; Pang, W.; Wu, X.; Jun, G.; Romero, L.; Taka, E.; Marculescu, D.; Nowatzki, T.; Vasireddy, P.; Melber, J.; et al. Can Asymmetric Tile Buffering Be Beneficial? *arXiv* **2025**, arXiv:2511.16041.
35. Hovhannisyan, A. Optimizing DGEMM Using Vectorized Micro-Kernels and Memory-Aware Parallelization. In Proceedings of the Computer Science and Information Technologies (CSIT) Workshop, CSIT 2025, London, UK, 26–27 July 2025. https://doi.org/10.51408/csit2025_81.
36. Zhang, Z.; Wang, H.; Xu, H.; Yang, D.; Zhou, X.; Cheng, D. HyTiS: Hybrid Tile Scheduling for GPU GEMM with Enhanced Wave Utilization and Cache Locality. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '25, St. Louis, MO, USA, 16–21 November 2025; pp. 1604–1618. <https://doi.org/10.1145/3712285.3759771>.
37. Rivera, C.; Chen, J.; Xiong, N.; Zhang, J.; Song, S.L.; Tao, D. TSM2X: High-performance tall-and-skinny matrix-matrix multiplication on GPUs. *J. Parallel Distrib. Comput.* **2021**, *151*, 70–85. <https://doi.org/10.1016/j.jpdc.2021.02.013>.
38. Tang, H.; Komatsu, K.; Sato, M.; Kobayashi, H. Efficient Mixed-Precision Tall-and-Skinny Matrix-Matrix Multiplication for GPUs. *Int. J. Netw. Comput.* **2021**, *11*, 267–282. https://doi.org/10.15803/ijnc.11.2_267.
39. Park, G.; Park, B.; Kim, M.; Lee, S.; Kim, J.; Kwon, B.; Kwon, S.J.; Kim, B.; Lee, Y.; Lee, D. LUT-GEMM: Quantized Matrix Multiplication based on LUTs for Efficient Inference in Large-Scale Generative Language Models. *arXiv* **2024**, arXiv:2206.09557.
40. Heo, G.; Lee, S.; Cho, J.; Choi, H.; Lee, S.; Ham, H.; Kim, G.; Mahajan, D.; Park, J. NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, La Jolla, CA, USA, 27 April–1 May 2024; Volume 3, pp. 722–737. <https://doi.org/10.1145/3620666.3651380>.

41. Hu, H.; Xiao, B.; Sun, S.; Yin, J.; Zhang, Z.; Luo, X.; Jiang, C.; Xu, W.; Jia, X.; Liu, X.; et al. LiquidGEMM: Hardware-Efficient W4A8 GEMM Kernel for High-Performance LLM Serving. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '25, St. Louis, MO, USA, 16–21 November 2025; pp. 1619–1630. <https://doi.org/10.1145/3712285.3759852>.
42. Sadasivan, H.; Ozturk, M.E.; Osama, M.; Millette, C.; Rai, A.; Podkorytov, M.; Afaganis, J.; Huang, C.; Zhang, J.; Liu, J. Stream-K++: Adaptive GPU GEMM Kernel Selection and Scheduling for AI Using Bloom Filters. In *High Performance Computing*; Neuwirth, S., Paul, A.K., Weinzierl, T., Carson, E.C., Eds.; Springer: Cham, Switzerland, 2026; pp. 480–493.
43. Taka, E.; Roesti, A.; Melber, J.; Vasireddy, P.; Denolf, K.; Marculescu, D. Striking the Balance: GEMM Performance Optimization Across Generations of Ryzen AI NPUs. *arXiv* **2025**, arXiv:2512.13282.
44. Huawei. Non-Contiguous-to-Contiguous Conversion (Vector Operators)-Basic Tuning-Operator Computation Perform. *Huawei*, 7 March 2024.
45. Huawei. AI Core-Background Knowledge-TBE&AI CPU Operator Development-Operator development-7.0.0-CANN commercial edition-Ascend Documentation-Ascend Community. *Huawei*, 6 February 2024.
46. Huawei. Hardware Architecture-Operator development-8.0.RC2.alpha003-CANN community edition-Ascend Documentation-Ascend Community. *Huawei*, 25 June 2024. (In Chinese)
47. Anderson, A.; Vasudevan, A.; Keane, C.; Gregg, D. High-Performance Low-Memory Lowering: GEMM-based Algorithms for DNN Convolution. In Proceedings of the 2020 IEEE 32nd International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD), Hilo, HI, USA, 13–15 November 2020; pp. 99–106. <https://doi.org/10.1109/SBAC-PAD49847.2020.00024>.
48. Han, Q.; Hu, Y.; Yu, F.; Yang, H.; Liu, B.; Hu, P.; Gong, R.; Wang, Y.; Wang, R.; Luan, Z.; et al. Extremely Low-bit Convolution Optimization for Quantized Neural Network on Modern Computer Architectures. In Proceedings of the 49th International Conference on Parallel Processing, ICPP '20, Edmonton, AB, Canada, 17–20 August 2020. <https://doi.org/10.1145/3404397.3404407>.
49. Yang, Z.; Lu, L.; Wang, R. A batched GEMM optimization framework for deep learning. *J. Supercomput.* **2022**, *78*, 13393.
50. Nath, R.; Tomov, S.; Dongarra, J. An Improved Magma Gemm For Fermi Graphics Processing Units. *Int. J. High Perform. Comput. Appl.* **2010**, *24*, 511–515. <https://doi.org/10.1177/1094342010385729>.
51. Huawei. Atlas 300T A2 Training Card User Guide 03. Available online: <https://support.huawei.com/enterprise/en/doc/EDOC1100338863/5549b5ec/performance?idPath=23710424|251366513|22892968|252309113|254184749> (accessed on 24 October 2023).
52. Nvidia. NVIDIA A800 40GB Active Graphics Card. Available online: <https://www.nvidia.com/en-us/products/workstations/a800> (accessed on 24 October 2023).
53. Zhang, S.; Roller, S.; Goyal, N.; Artetxe, M.; Chen, M.; Chen, S.; Dewan, C.; Diab, M.; Li, X.; Lin, X.V.; et al. OPT: Open Pre-trained Transformer Language Models. *arXiv* **2022**, arXiv:2205.01068.
54. Dongarra, J.; Luszczek, P. HPL-MxP benchmark: Mixed-precision algorithms, iterative refinement, and scalable data generation. *Int. J. High Perform. Comput. Appl.* **2025**. <https://doi.org/10.1177/10943420251382476>.
55. Xue, W.; Yang, K.; Liu, Y.; Fan, D.; Xu, P.; Tian, Y. Unlocking High Performance with Low-Bit NPUs and CPUs for Highly Optimized HPL-MxP on Cloud Brain II. In Proceedings of the SC24: International Conference for High Performance Computing, Networking, Storage and Analysis, Atlanta, GA, USA, 17 November 2024; pp. 1–16. <https://doi.org/10.1109/SC41406.2024.00088>.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.